

LABORATORY OBSERVATION / RECORD

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: 6

Student Name: K. Sirisha

Roll No.: A24126552260

Section: B

NPM AND package.json IN NODE.JS (CO-2)

1. TITLE

Explain NPM and package.json in Node.js. Describe the Steps Involved in Installing an External Package and Using It in a Node.js Application.

2. AIM

To study the Node Package Manager (NPM) and the package.json file, and to demonstrate the steps of creating a Node.js project, installing an external package, and using it in an application.

3. OBJECTIVE

- To understand what NPM is and the role of the npm registry.
- To understand the structure and the important fields of package.json.
- To create a project using npm init and to manage packages using npm install and npm uninstall.
- To differentiate between dependencies and devDependencies, and between local and global installation.
- To understand the purpose of node_modules, package-lock.json, and semantic versioning.
- To install an external package (lodash) and to use it in a Node.js application using require().
- To use important NPM commands such as npm list and npm start.

4. THEORY

1. What is NPM?

NPM (Node Package Manager) is the default package manager of Node.js and is installed automatically along with it. It has three parts: (1) the **npm registry**, a huge online database of reusable open-source packages (npmjs.com); (2) the **npm CLI**, the command-line tool used to install, update, and remove packages; and (3) the npm website. Instead of writing every feature from scratch, developers reuse packages such as express, lodash, or mongoose. A **package** is a folder that contains code and a package.json file describing it.

2. What is package.json?

package.json is a JSON file placed in the root folder of a Node.js project. It stores the metadata of the project and the list of packages on which the project depends. It is created using `npm init`. Anyone who receives the project can rebuild the installation by running `npm install`, so the `node_modules` folder does not have to be shared.

Field	Purpose
name	Name of the project (lower case, no spaces).
version	Version of the project in the form MAJOR.MINOR.PATCH, for example 1.0.0.
description	Short description of the project.
main	Entry-point file of the project, for example app.js.
scripts	Named commands run with <code>npm run <name></code> . The names start and test can be run as <code>npm start</code> and <code>npm test</code> .
dependencies	Packages that the application needs in order to run.
devDependencies	Packages needed only during development or testing, for example nodemon or jest.
keywords, author, license	Information used when the package is published, for example license ISC or MIT.

3. node_modules and package-lock.json

`node_modules` is the folder created by NPM in which the downloaded packages (and the packages on which they depend) are stored. It can be very large, so it is normally not shared or uploaded to Git; it can always be recreated by running `npm install`. `package-lock.json` records the exact version of every installed package so that every developer gets an identical installation.

4. Semantic Versioning

A package version is written as MAJOR.MINOR.PATCH (for example 4.17.21). MAJOR changes may break existing code, MINOR changes add features in a compatible way, and PATCH changes fix bugs. In package.json, `^4.17.21` allows updates to any newer minor or patch version below 5.0.0, `~4.17.21` allows only newer patch versions, and `4.17.21` allows exactly that version.

5. Local and Global Installation, Dependencies and devDependencies

- **Local installation** (default): `npm install lodash` stores the package in the `node_modules` folder of the current project, and it is recorded under `dependencies`.
- **Global installation**: `npm install -g nodemon` installs the package once for the whole system; it is used mainly for command-line tools.
- **Development dependency**: `npm install jest --save-dev` records the package under `devDependencies` because it is not needed when the application is running in production.

6. Common NPM Commands

Command	Purpose
npm init / npm init -y	Creates package.json (with questions / with default values).
npm install <package>	Installs a package locally and adds it to dependencies.
npm install	Installs all the packages listed in package.json.
npm install <package> --save-dev	Installs a package as a development dependency.
npm install -g <package>	Installs a package globally.
npm uninstall <package>	Removes a package and updates package.json.
npm update	Updates the packages within the version range given in package.json.
npm list	Displays the installed packages in the form of a tree.
npm outdated	Shows the packages that have newer versions available.
npm start / npm run <name>	Runs the script defined under scripts in package.json.

7. Steps to Install and Use an External Package

- Install Node.js and check the installation using `node -v` and `npm -v`.
- Create a project folder and run `npm init -y` to create package.json.
- Run `npm install <package>`. NPM downloads the package from the registry into `node_modules`, creates `package-lock.json`, and adds an entry under dependencies.
- Load the package in the code using `require('<package>')` and call its functions.
- Run the application using `node app.js` or `npm start`.

```
npm install lodash
      |
      v
npm registry (npmjs.com) --- download ---> node_modules/lodash
      |                                   |
      +--> package.json ("dependencies")  +--> used in code by
      +--> package-lock.json (exact versions)  require('lodash')
```

5. ALGORITHM / PROCEDURE

1. Start.
2. Open the terminal and verify the installation using `node -v` and `npm -v`.
3. Create a folder named `npm-demo` and move into it (`mkdir npm-demo`, then `cd npm-demo`).
4. Run `npm init -y` to generate package.json with default values.
5. Edit package.json: set main to `app.js`, add the start script, and set the author name.
6. Install the external package using `npm install lodash`. Observe that the `node_modules` folder and `package-lock.json` are created and that `lodash` is added under dependencies.

7. Create app.js and load the package using `const _ = require('lodash')`.
8. Use the lodash functions `uniq`, `orderBy`, `sum`, `mean`, `max`, `min`, `chunk`, `groupBy`, `maxBy`, `map`, and `capitalize` on sample data.
9. Run the application using `node app.js` (or `npm start`).
10. Run `npm list` to view the installed package.
11. (Optional) Remove the package using `npm uninstall lodash` and observe the change in `package.json`.
12. Stop.

6. PROGRAM

Terminal commands

```
node -v
npm -v
mkdir npm-demo
cd npm-demo
npm init -y
npm install lodash
node app.js
npm list
```

package.json (after installing lodash)

```
{
  "name": "npm-demo",
  "version": "1.0.0",
  "description": "Experiment 7 - NPM and package.json demonstration",
  "main": "app.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1",
    "start": "node app.js"
  },
  "keywords": [],
  "author": "K. Sirisha",
  "license": "ISC",
  "dependencies": {
    "lodash": "^4.17.21"
  }
}
```

app.js

```
// app.js - uses the external package 'lodash', which was installed through NPM

// Step 1: Load the installed package
const _ = require('lodash');

// Step 2: Helper function to print a label and a value in aligned form
const show = (label, value) => console.log(label.padEnd(20) + ':', value);

// Step 3: Use lodash functions on an array of numbers
const marks = [45, 78, 92, 67, 78, 92];
show('Marks', marks);
show('Unique marks', _.uniq(marks));
show('Sorted (highest 1st)', _.orderBy(marks, [], ['desc']));
show('Sum / Average', `${_.sum(marks)} / ${_.mean(marks).toFixed(2)}`);
show('Max / Min', `${_.max(marks)} / ${_.min(marks)}`);
show('Groups of 2', _.chunk(marks, 2));

// Step 4: Create an array of objects
const students = [
  { name: 'Anitha', dept: 'CSE', marks: 88 },
  { name: 'Ravi', dept: 'ECE', marks: 72 },
  { name: 'Kiran', dept: 'CSE', marks: 91 },
  { name: 'Meena', dept: 'ECE', marks: 65 },
];

// Step 5: Use lodash functions on the array of objects
```

```
show('Names by department',  
  _.mapValues(_.groupBy(students, 'dept'), g => _.map(g, 'name')));  
show('Top scorer', _.maxBy(students, 'marks').name);  
show('All names', _.map(students, 'name'));  
  
// Step 6: Use a lodash string function  
show('Capitalised text', _.capitalize('NODE.JS PACKAGE MANAGER'));
```

7. CODE EXPLANATION

Code	Explanation
<code>npm init -y</code>	Creates package.json with default values without asking any questions.
<code>"main": "app.js"</code>	Entry-point file of the project.
<code>"scripts": { "start": "node app.js" }</code>	Defines a command that runs with npm start.
<code>"dependencies": { "lodash": "^4.17.21" }</code>	Lists the external package needed by the application. The ^ allows compatible minor and patch updates. NPM adds this entry automatically during npm install lodash.
<code>npm install lodash</code>	Downloads lodash from the npm registry into node_modules and updates package.json and package-lock.json.
<code>const _ = require('lodash');</code>	Loads the installed package. Packages are loaded by name, and Node.js looks for them in node_modules.
<code>const show = (label, value) => ...</code>	Small helper function that prints a label (aligned using padEnd) and its value.
<code>_.uniq(marks)</code>	Removes duplicate values from the array.
<code>_.orderBy(marks, [], ['desc'])</code>	Sorts the array in descending order.
<code>_.sum(marks), _.mean(marks)</code>	Total and average of the values.
<code>_.max(marks), _.min(marks)</code>	Highest and lowest values.
<code>_.chunk(marks, 2)</code>	Splits the array into groups of 2 elements.
<code>_.groupBy(students, 'dept')</code>	Groups the objects by the dept property. _.mapValues and _.map then keep only the names.
<code>_.maxBy(students, 'marks')</code>	Returns the object that has the highest marks.
<code>_.capitalize(text)</code>	Converts the first letter to upper case and the remaining letters to lower case.
<code>npm list</code>	Displays the installed packages in a tree form.

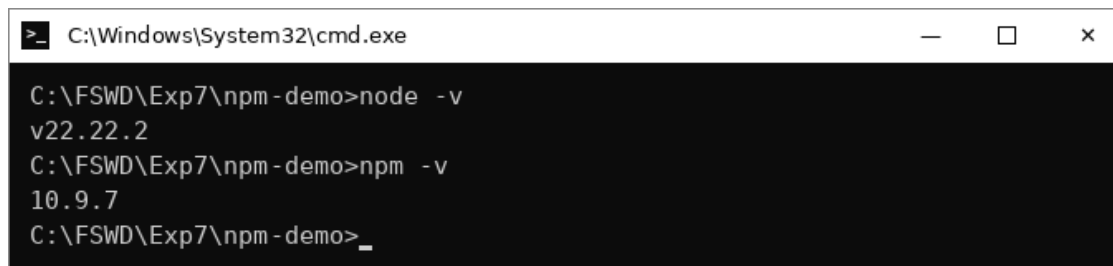
8. TEST CASES AND EXECUTION DATA

The following test cases were recorded after executing the commands and the application in a terminal.

Test Case TC-01

Test / Input	Run <code>node -v</code> and <code>npm -v</code> in the terminal.
Expected Result	The version numbers of Node.js and NPM are displayed, which confirms that both are installed.
Actual Result	Both version numbers were displayed, so Node.js and NPM are installed and working.
Status	Pass

TC-01 Output:



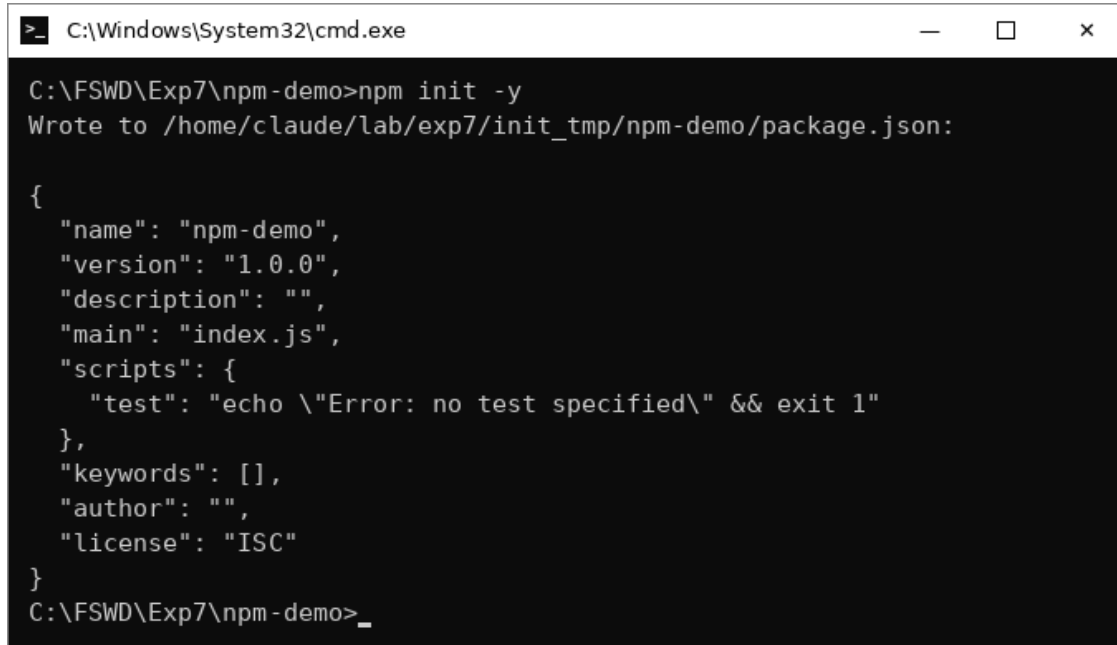
```
> C:\Windows\System32\cmd.exe

C:\FSWD\Exp7\npm-demo>node -v
v22.22.2
C:\FSWD\Exp7\npm-demo>npm -v
10.9.7
C:\FSWD\Exp7\npm-demo>_
```

Test Case TC-02

Test / Input	Create the folder npm-demo, move into it, and run <code>npm init -y</code> .
Expected Result	package.json is created with default values (name, version, main, scripts, author, license) and its content is displayed.
Actual Result	NPM created package.json and displayed its content: name npm-demo, version 1.0.0, main index.js, and license ISC.
Status	Pass

TC-02 Output:

A screenshot of a Windows command prompt window titled "C:\Windows\System32\cmd.exe". The prompt shows the command "C:\FSWD\Exp7\npm-demo>npm init -y" and its output. The output indicates that a package.json file was created at a specific path and displays its contents in JSON format. The JSON includes fields for name, version, description, main, scripts, keywords, author, and license. The "test" script is set to echo an error message and exit with a status of 1. The prompt ends with "C:\FSWD\Exp7\npm-demo>_".

```
>_ C:\Windows\System32\cmd.exe

C:\FSWD\Exp7\npm-demo>npm init -y
Wrote to /home/claude/lab/exp7/init_tmp/npm-demo/package.json:

{
  "name": "npm-demo",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC"
}
C:\FSWD\Exp7\npm-demo>_
```

Test Case TC-03

Test / Input	Run <code>npm install lodash</code> inside the npm-demo folder (internet connection required).
Expected Result	lodash is downloaded from the registry into node_modules, package-lock.json is created, NPM prints a summary such as "added 1 package", and package.json now contains lodash under dependencies.
Actual Result	To be written after running the command on your system.
Status	Pending - attach screenshot

TC-03 Output:

PASTE YOUR SCREENSHOT HERE

Run: `npm install lodash` (then type `package.json` / `cat package.json`)

Capture the terminal showing the install summary and the dependencies entry.

Test Case TC-04

Test / Input	Run node app.js after editing package.json and creating app.js.
Expected Result	The lodash functions process the sample data: unique marks, descending order, sum and average, maximum and minimum, groups, department-wise names, top scorer, and the capitalised text.
Actual Result	The application ran without errors and displayed all results, for example sum / average 452 / 75.33, top scorer Kiran, and the text "Node.js package manager".
Status	Pass

TC-04 Output:

```
C:\Windows\System32\cmd.exe

C:\FSWD\Exp7\npm-demo>node app.js
Marks                : [ 45, 78, 92, 67, 78, 92 ]
Unique marks         : [ 45, 78, 92, 67 ]
Sorted (highest 1st): [ 92, 92, 78, 78, 67, 45 ]
Sum / Average        : 452 / 75.33
Max / Min            : 92 / 45
Groups of 2          : [ [ 45, 78 ], [ 92, 67 ], [ 78, 92 ] ]
Names by department  : { CSE: [ 'Anitha', 'Kiran' ], ECE: [ 'Ravi', 'Meena' ] }
Top scorer           : Kiran
All names             : [ 'Anitha', 'Ravi', 'Kiran', 'Meena' ]
Capitalised text     : Node.js package manager
C:\FSWD\Exp7\npm-demo>
```

Test Case TC-05

Test / Input	Run npm list.
Expected Result	A tree is displayed with the project npm-demo@1.0.0 and the installed package lodash.
Actual Result	The project npm-demo@1.0.0 was displayed with lodash listed as its installed dependency.
Status	Pass

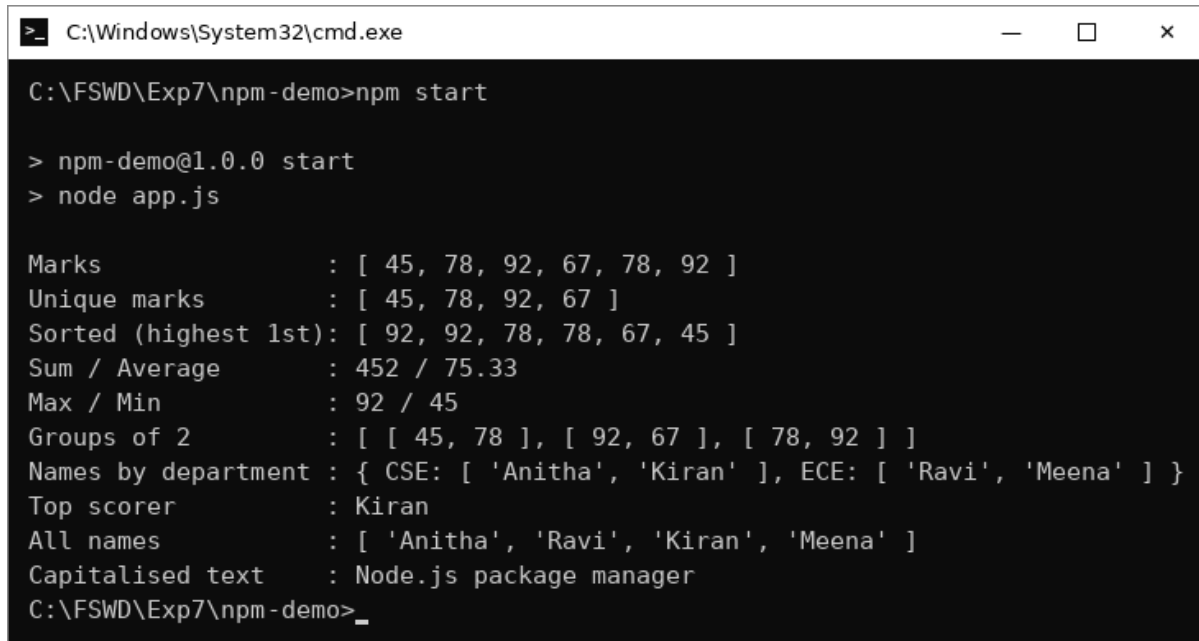
TC-05 Output:

```
C:\Windows\System32\cmd.exe
```

```
C:\FSWD\Exp7\npm-demo>npm list
npm-demo@1.0.0 /home/claude/lab/exp7/npm-demo
├─ lodash@4.18.1
C:\FSWD\Exp7\npm-demo>
```

9. OUTPUT

Output 1: Execution of the application using npm start



```
C:\Windows\System32\cmd.exe
C:\FSWD\Exp7\npm-demo>npm start

> npm-demo@1.0.0 start
> node app.js

Marks           : [ 45, 78, 92, 67, 78, 92 ]
Unique marks    : [ 45, 78, 92, 67 ]
Sorted (highest 1st): [ 92, 92, 78, 78, 67, 45 ]
Sum / Average   : 452 / 75.33
Max / Min       : 92 / 45
Groups of 2     : [ [ 45, 78 ], [ 92, 67 ], [ 78, 92 ] ]
Names by department : { CSE: [ 'Anitha', 'Kiran' ], ECE: [ 'Ravi', 'Meena' ] }
Top scorer      : Kiran
All names       : [ 'Anitha', 'Ravi', 'Kiran', 'Meena' ]
Capitalised text  : Node.js package manager
C:\FSWD\Exp7\npm-demo>_
```

Figure 1: Output of the application that uses the external package lodash

10. RESULT

Thus, **NPM** and the **package.json** file were studied, and a Node.js project was created using npm init. The external package lodash was installed using npm install, recorded under dependencies in package.json, loaded using require(), and used successfully in the application. The experiment demonstrated how NPM manages packages, versions, and scripts in a Node.js project.